

Especificação Técnica e de Processos Avançados: Backend & Motor SafeID

Este documento detalha exaustivamente a arquitetura de software, os fluxos operacionais, os algoritmos matemáticos e a infraestrutura do ecossistema SafeID. O foco é garantir a construção de um sistema de nível corporativo, com alta disponibilidade, resiliência contra falhas em cascata, escalabilidade horizontal e estrita conformidade com as melhores práticas de cibersegurança e a Lei Geral de Proteção de Dados (LGPD).

1. Fundamentação e Arquitetura de Software (Clean Architecture)

O backend adota o padrão **Clean Architecture** (Arquitetura Limpa), promovendo um altíssimo grau de desacoplamento. Isso garante que o "Core" (Motor de Risco) não dependa do framework web (Express) ou do banco de dados (PostgreSQL).

1.1. Estrutura de Diretórios e Domínios

/safeid-backend	
├─ /src	
│ ├─ /@types	# Sobrescritas e interfaces TypeScript globais
│ ├─ /api	# Camada de Apresentação (Interface de Comunicação)
│ │ ├─ /controllers	# Controladores REST (Injetam os Use Cases)
│ │ ├─ /middlewares	# Filtros de contexto (RateLimit, Helmet, Auth, ErrorHandler)
│ │ └─ /routes	# Definição de endpoints e mapeamento de verbos HTTP
│ └─ /core	# Camada de Domínio e Aplicação (O "Coração" do SafeID)
│ ├─ /engines	# Lógica pesada: RiskEngine (Matemática) e AIEngine (Cognição)
│ ├─ /entities	# Entidades de negócio (Ex: User, ScanResult)
│ ├─ /repositories	# Interfaces de contratos para o Banco de Dados
│ └─ /use-cases	# Regras de orquestração (Ex: ExecuteRiskScanUseCase)
│ └─ /infra	# Camada de Infraestrutura (Implementações Externas)
│ ├─ /cache	# Client do Redis (ioredis) e estratégias de eviction
│ ├─ /database	# Implementação Prisma ORM e PgBouncer
│ ├─ /http	# Clientes HTTP externos (Axios configurado para a HIB)
│ └─ /queue	# Configurações BullMQ (Producers, Workers e Queue Events)
│ └─ /shared	# Recursos transversais
│ ├─ /utils	# Helpers genéricos (Sanitizers, UUID generators)
│ ├─ /crypto	# Módulo proprietário de criptografia (AES-256-GCM)
│ └─ /schemas	# Contratos de validação com Zod
└─ server.ts	# Entry point de bootstrap (Graceful Shutdown)
├─ /prisma	# Schema DSL e histórico de Migrations
├─ /tests	# Suíte Completa de QA
│ ├─ /unit	# Testes dos Engines e Use Cases (sem I/O)
│ ├─ /integration	# Testes de Rotas com Banco de Dados em Memória / Test
│ └─ /e2e	# Testes ponta a ponta
├─ docker-compose.yml	# Orquestração de containers local
├─ Dockerfile	# Definição de imagem Multi-stage build
└─ .env	# Environment variables injection

2. Tecnologias de Base e Comportamento

2.1. Runtime: Node.js (V8 Engine) e TypeScript

- **Funcionamento:** O Node.js opera de forma assíncrona orientada a eventos (Event Loop). Em um sistema focado em consultas externas (HIBP e OpenAI), 90% do tempo de resposta é gasto aguardando I/O de rede. O Node.js permite que a *Thread* principal continue aceitando novas requisições enquanto aguarda essas APIs externas, otimizando o consumo de CPU.
- **TypeScript:** Utilizado no modo `strict: true`. Garante que interfaces de respostas externas (JSONs do HIBP) sejam previamente tipadas, evitando erros de *undefined* em tempo de execução.

2.2. Web Framework: Express.js + Hardening

O Express atuará não apenas como roteador, mas como um escudo primário.

- **Helmet.js:** Middleware ativado para remover headers reveladores (como `X-Powered-By`) e injetar proteções como `Strict-Transport-Security` (HSTS), `X-Frame-Options` (contra Clickjacking) e `X-Content-Type-Options`.
- **Express-Rate-Limit:** Configurado com uma janela deslizante (Sliding Window). Visitantes não autenticados têm limite de **5 requisições a cada 15 minutos** por IP. Se o limite for excedido, o backend retorna `429 Too Many Requests`.

3. Ciclo de Vida da Requisição (Pipeline de Dados)

3.1. Triagem e Sanitização Rigorosa (Zod)

Quando um e-mail chega via `POST /api/v1/scan`, ele não toca no banco de dados antes de passar pelo funil do Zod:

1. **Tipagem:** Valida se é `string`.
2. **Formato RFC:** Verifica se atende aos padrões de um e-mail válido.
3. **Comprimento:** Rejeita strings com mais de 255 caracteres (prevenção de Buffer Overflow).
4. **Filtro de Descartáveis (Opcional):** Verifica se o domínio do e-mail não pertence a provedores de e-mails temporários (ex: Mailinator), bloqueando consultas inúteis.
5. **Normalização:** Aplica `toLowerCase()` e um parser Unicode, prevenindo ataques de homógrafos.

3.2. Cache de Alta Performance (Redis)

O Redis (operando via biblioteca `ioredis`) é a primeira barreira de consulta para aliviar o backend e economizar a cota da API HIBP.

- **Hash de Chave:** O e-mail não é salvo em texto claro no Redis. O sistema aplica um hash SHA-256 no e-mail (`scan:{sha256(email)}`).
- **Política de TTL (Time To Live):**
 - Conta Comprometida: O cache dura **12 horas**.

- #### 4. O Motor de Orquestração OSINT (BullMQ & Circuit Breaker)

4.1. Fila de Processamento Distribuído (BullMQ)

1. **Producer:** O e-mail (hasheado) entra em uma fila do BullMQ no Redis.
2. **Worker Concurrency Limit:** O Worker é instanciado com concorrência 1 e `limiter: { max: 1, duration: 1500 }`.
3. **Mecânica de Polling/Retorno:** O Controller Express aguarda a resolução do Job (timeout máximo de 10 segundos).
4. **Stalled Jobs e Retry:** Se a rede oscilar, o BullMQ tenta novamente a mesma consulta até 3 vezes, utilizando um **Exponential Backoff** (espera 2s, 4s, 8s).

Caso a HIBP fique fora do ar globalmente, tentar enviar Jobs para a fila é um desperdício.

- **Métricas:** O `Opossum` envolve o cliente HTTP.
- **Regra de Disparo:** Se `errorThresholdPercentage` passar de 50% em uma janela de 10 segundos (mínimo de 5 chamadas), o circuito entra em **Estado Aberto (Open)**.
- **Comportamento:** O backend rejeita imediatamente novas consultas informando "Manutenção do Provedor" (Fail Fast), protegendo as threads do Node.js. Após 30 segundos, ele entra em **Half-Open** e deixa passar 1 requisição de teste. Se sucesso, fecha o circuito.

5. O Cérebro: Motor de Cálculo de Risco (Risk Engine)

5.1. Dicionário Hierárquico de Dados (Data Classes)

Atribuição de pesos baseados no potencial destrutivo do dado exposto:

Categoria	Tipos de Dados (Exemplos do HIBP)	Peso (P)	Justificativa
-----------	-----------------------------------	--------------	---------------

Nível 1 (Crítico)	Passwords, Banking Information, Credit Cards, SSN	10	Risco de fraude financeira e Account Takeover imediato.
Nível 2 (Alto)	Phone numbers, Physical addresses, Passports	7	Permite SIM Swapping, extorsão e roubo de identidade física.
Nível 3 (Médio)	Names, DOB, Job Titles, Employers	4	Facilita ataques de Engenharia Social elaborados (Spear Phishing).
Nível 4 (Baixo)	Email addresses, Usernames, IP addresses	1	Risco de SPAM ou Phishing genérico.

5.2. Fórmula Matemática de Avaliação Ponderada

Para cada vazamento i , calcula-se o Sub-Score (S_i):

$$S_i = \max(P_{classes}) \times F_{recencia} \times F_{confianca}$$

Onde:

- $\max(P_{classes})$: O motor não soma todos os dados do mesmo vazamento. Ele pega o **dado mais crítico** vazado. (Ex: Se vazou Nome e Senha, o peso base é 10).
- $F_{recencia}$ **(Fator de Recência)**:
 - $\Delta t < 12$ meses: Multiplicador de **1.5** (Risco agudo, senha provavelmente ativa).
 - $12 \leq \Delta t \leq 36$ meses: Multiplicador de **1.0**.
 - $\Delta t > 36$ meses: Multiplicador de **0.6** (Maioria dos usuários já rotacionou credenciais).
- $F_{confianca}$ **(Fator de Verificação)**:
 - Vazamento verificado pelo HIBP (`isVerified: true`): **1.0**
 - Vazamento não confirmado (unverified) ou spam list: **0.5**

5.3. Agregação e Logaritmo de Suavização

O Risk Score final (RS_{final}) é agregado somando os Sub-Scores. Para evitar que 20 vazamentos pequenos gerem um score absurdo, aplicamos uma função de suavização ou um teto estrito (`Math.min(ScoreTotal, 100)`).

- Classificação:
 - 0 - 30 : Risco Baixo (Verde)
 - 31 - 70 : Risco Moderado (Amarelo)
 - 71 - 100 : Risco Crítico (Vermelho)

6. Módulo de Inteligência Cognitiva (OpenAI Engine)

Transforma a análise estruturada em orientações semânticas focadas na **Literacia Digital**.

6.1. Configurações da API OpenAI

- **Modelo:** gpt-4o-mini (Menor latência e custo, ideal para sumarização estruturada).
- **Temperature:** 0.2 (Garante respostas técnicas, determinísticas e diretas, reduzindo alucinações).
- **Max Tokens:** Limitado a 300 tokens para evitar respostas longas e manter a objetividade exigida pelo usuário leigo.
- **Response Format:** type: "json_object" .

6.2. Estratégia de Prompting e Sanitização

Extrema Segurança: O e-mail e qualquer outro dado rastreável **NUNCA** são enviados no payload da requisição para a OpenAI. *Payload gerado pelo backend:*

```
{
  "context": "Vazamento no serviço Canva ocorrido há 2 anos.",
  "exposed_data": ["Passwords", "Email addresses", "Names"]
}
```

A IA processa esse metadado genérico e retorna um JSON contendo chaves como `executive_summary` e `mitigation_steps` em português claro.

7. Persistência de Dados e LGPD (PostgreSQL + Prisma)

7.1. Criptografia em Repouso (AES-256-GCM)

Para o armazenamento do histórico de usuários autenticados (PI-2), a aplicação obedece ao princípio da Privacidade por Design.

- **Módulo nativo** `crypto` : Antes de salvar qualquer identificador no PostgreSQL, o Prisma dispara um middleware que criptografa a string.
- **Mecânica:** Utiliza AES-256-GCM (Autenticação Galois/Counter Mode). O algoritmo exige uma Chave Mestra (Secreta) e gera um Vetor de Inicialização (IV) aleatório e uma Tag de Autenticação para cada registro, impedindo adulterações no banco.
- **Descriptografia:** Feita apenas em memória (RAM) no momento em que o usuário dono do dado realiza o login e solicita seu histórico.

7.2. Relacionamento de Entidades (Prisma Schema)

- Tabela `User` : Contém hash de senha de acesso (`Argon2id` ou `Bcrypt`) e e-mail criptografado.
- Tabela `ScanHistory` : Ligada ao usuário. Armazena apenas UUID, ID do vazamento, Data e Risk Score numérico (Nunca armazena o JSON bruto do vazamento).

8. Observabilidade, Monitoramento e Logs

Sem observabilidade, o backend opera às cegas.

- **Winston Logger:** Configurado para exportar logs no formato JSON.
 - *Nível info* : Requisições processadas, tempos de fila do BullMQ.
 - *Nível warn* : Circuit Breaker aberto, Rate Limit excedido.
 - *Nível error* : Falhas na OpenAI, Timeout na HIBP (com stack trace mascarado para não expor variáveis de ambiente).
- **Health Checks (Liveness e Readiness):** O endpoint `/api/health` retorna `200 OK` apenas se o Node.js consegue fazer `PING` no PostgreSQL e no Redis, garantindo que o orquestrador (Docker) saiba a saúde da aplicação.

9. QA, Testes e Integração Contínua (CI/CD)

9.1. Pirâmide de Testes

1. **Unitários (Jest):** Testa o `RiskEngine` exaustivamente. É obrigatório criar um teste com um objeto de 10 vazamentos mistos para garantir que o cálculo matemático resulta no score exato esperado.
2. **Integração com Nock:** O `Nock` intercepta qualquer chamada HTTP para fora. Ele forja respostas da HIBP.
 - *Teste de Resiliência:* Simula a HIBP retornando HTML quebrado e verifica se o backend captura o erro com elegância usando blocos `try/catch`.
3. **Property-Based Testing:** Envia milhares de strings malucas para o sistema garantir que a validação Zod não tenha "furos".

9.2. GitHub Actions (CI Pipeline)

Sempre que um Pull Request é aberto para a branch `main` :

1. Sobe containers de banco e cache descartáveis.
2. Roda a verificação estática do código (`ESLint` e `Prettier`).
3. Compila o TypeScript para JavaScript (verificando erros tipográficos).
4. Executa 100% da suíte de testes.
5. Se a cobertura de código (Coverage) for menor que 80%, bloqueia o PR.

10. Infraestrutura em Contêineres (Docker Multi-stage)

A aplicação é empacotada em uma imagem Docker utilizando **Multi-stage Build** para manter o peso leve e seguro.

1. **Fase Builder:** Baixa todas as dependências do `package.json`, instala o Prisma Client, e compila de `.ts` para `.js`.
2. **Fase Runner:** Copia apenas o código JavaScript compilado e dependências de produção para uma imagem `node:18-alpine` limpa.

3. Orquestração (`docker-compose.yml`):

- Cria uma rede virtual interna (`safeid-network`).
- Bloqueia portas do Banco e Redis para o exterior, expondo apenas a porta `3000` da API Node.js para comunicação com o Frontend React.

Documento de Especificação de Engenharia Backend - SafeID *Este documento é a fonte de verdade para todas as decisões arquiteturais e desenvolvimento de lógica do servidor.*